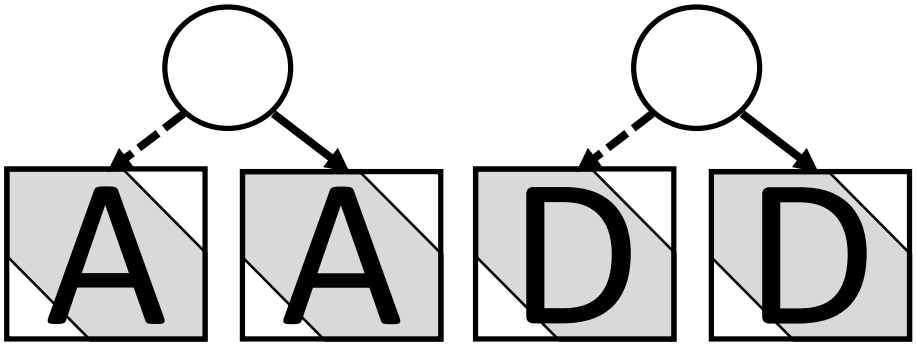
AADD Multi-Platform User's Guide
Version 0.9



University Kaiserslautern Landau
Chair of Cyber-Physical Systems
Hagen Heermann, Christoph Grimm, Carna Zivkovic

The AADD library is a library for computing with ranges. It is developed at the University of Kaiserslautern-Landau with support from several research projects. We in particular want thank the following funding agencies and companies that provide and provided financial support:

- ANCONA; BMBF, Förderkennzeichen 16ES0210-15.
- Robert Bosch AG, Germany.
- Mentor Graphics GmbH, Germany.
- Intel AG, Germany.
- AVL List, Graz, Austria.
- MOVE; German/French DFG project.
- GENIAL!; BMBF F&E 16ES0865-16ES0876.
- Arrowhead Tools; ECSEL Joint Undertaking (JU) under grant agreement No 82645.
- AnastASICa, BMBF F&E.
- AVL List, Graz, Austria.

Contents

1 An introduction to the AADD Library

1.1 Why, Why, Why?

Computing with numbers is one of the prime use cases for computers. However, number representations are often not accurate, and, even more, often it is very inefficient to compute with single values and trying all combinations of values. In use cases like *Reachability Analysis, Constraint propagation, Formal Verification, Reachability analysis*, etc., one uses sets of possible values and defines operations that compute on sets of possible values.

In this context, the AADD library provides means for

- representing sets of convex subsets like {1 .. 2, 50 .. 100, 200 .. 300},
- where the elements are Booleans, Integers, Reals, and
- computing set-operations like union, intersect and arithmetic or Boolean operations on the representations.

Main features of the AADD library include

- Implementation in Kotlin Multi-Platform for the JVM and various binary platforms,
- Availability on GitHub and Maven,
- Use of state-of-the-art approaches for achieving high accuracy and scalability,
- Unique combination with Decision Diagrams for taking profit of dependencies between Boolean and Arithmetic subproblems.

1.2 Intervals

Intervals are used for various reasons:

- *Representation of uncertain or unknown values.* Often upper and/or lower bounds of unknown quantities can be given.
- *A precise value cannot be represented.* Quantities such as π and other Real numbers cannot be represented precisely by e.g. floating-point numbers, no matter how many bits are used. Nevertheless, we can give upper and lower bounds: $3 < \pi < 4$ is mathematically precise, while $\pi = 3.14$ is wrong.
- *Multiple values are possible.* Often, we leave the precise value for a quantity in a specification open and just give lower and/or upper bounds.

A real-valued interval is a set of real numbers that is specified by an upper and lower bound:

$$[min, max] := \{min, max, x \in \mathbb{R} \mid \ min \leq x \leq max\}$$

We distinguish the following kind of intervals:

- *Empty* – The interval is the empty set {}, and $max < min$,
- *Scalar* – The interval has exactly one element: $[x, x] = \{x\}$, and $min = max$,
- *Finite* – The interval bounds are finite, and $min < max$,
- *Open* – One of the bounds is infinite,
- *Reals* – Both bounds are infinite, and the interval is the set $\mathbb{R}$ $(-\infty, \infty)$,

AADD library supports intervals of the kind *empty, scalar, finite, open,* and all *reals*. The AADD library supports intervals of Reals (RealRange, AADD), Integers (IntegerRange, IDD), Booleans (XBool, BDDa) and its interactions.

1.3 Computing with intervals

The nice thing with intervals is that we can easily do the arithmetic operations on the intervals as if they were real numbers and get an interval as result. Unfortunately, many laws for the reals do not hold for intervals. However, the following holds: let $f\colon \mathbb{R}^m \to \mathbb{R}$ be a function on the reals, and $F\colon \mathbb{I}^m \to \mathbb{I}$ is its interval-extension. Then, for $V \in \mathbb{I}^m$:

$$v \in V \Rightarrow f(v) \in F(V)$$

For example, if we add two intervals $[1,2]$, $[3,4]$ then the sum of two intervals $[4,6] = [1+3, 2+4]$ contains all sums of its real-valued elements.

Furthermore, for two intervals U, V, it holds:

$$U \subseteq V \Rightarrow F(U) \subseteq F(V)$$

For example, consider $[2,3]$ and $[1,4]$. Then $sqr([2,3])$ shall be subset of $sqr([1,4])$.

Note, that this not trivially holds for all functions and intervals, e.g. in the case of local extrema.

> AADD defines arithmetic operations on intervals of Reals and Integers, and Boolean operations on the Booleans.

In addition to classical "textbook" interval arithmetic, AADD library handles *overflows, rounding errors,* and *undefined operations*. An interval is generally interpreted as a Lattice with at least the set operations *join* and *intersect, and (in)equalities.* Sets can be from a domain (Reals, Integer, Booleans), and we distinguish

- *Empty* (also: Bottom); the empty set,
- *Zero*; a set with only the value 0 or False for Booleans,
- *One*; a set with only the value 1 or True for Booleans,
- *All* (also: Universe); the set with all elements from the domain.

1.4 Affine arithmetic

We generally allow some *over-approximation*. For example, we intuitively understand that

$$[1,2] + [3,4] = [4,6].$$

Unfortunately, for many functions, over-approximation cannot be avoided. A simple example where over-approximation can occur is the subtraction of two intervals:

$$[0,2] - [0,2] = [-2,2]$$

Intuitively, if we subtract a value from itself, we expect the result 0. Unfortunately, we do not know if the two intervals $[0,2]$ mean the same value, or two independent values from $[0,2]$; hence, the result – without knowing dependency – must be $[-2,2]$. In longer computational chains, over-approximations accumulate and eventually lead to an unbounded interval. To reduce this effect, AADD combines *affine arithmetic*[1] with several extensions. Affine arithmetic tracks linear dependencies in affine forms. An *affine form* $\tilde{x}$ is a linear combination of independent noise variables:

$$\tilde{x} = x_0 + x_1\varepsilon_1 + \cdots + x_n\varepsilon_n$$

where the coefficients x_i are floating point numbers, and the noise variables ε_i are unknown reals from the interval $[-1,1]$. Now, the interval $[0,2]$ can be represented as $1 + \varepsilon_1$; then we get:

$$(1 + \varepsilon_1) - (1 + \varepsilon_1) = 0$$

However, the second interval $[0,2]$ can also be represented as $1 + \varepsilon_2$; then we get

$$(1 + \varepsilon_1) - (1 + \varepsilon_2) = \varepsilon_1 - \varepsilon_2$$

[1] Stolfi J.; Figuiredo L.H.: An Introduction to Affine Arithmetic. Trends in Applied and Computational Mathematics, Vol. 4 No. 3 (2003), https://tema.sbmac.org.br/tema/article/view/352.

which describes an interval $[-2,2]$.

> Affine arithmetic represents *dependencies* among *different intervals* by the *index* of noise variables. Noise variables with the same index share a dependency; noise variables with different index are independent.

Linear operations on affine forms are (if computed with Reals) simple and accurate: For a sum, we just add the respective noise coefficients. For a product with a constant, we multiply the coefficients with the constant.

Nonlinear operations on affine forms are not simple and often not accurate. They first linearize the function which introduces a linearization error. The AADD library uses Taylor and Chebychev/MinMax methods to minimize the linearization error. Then, an affine form is built adds the linearization error with a new noise variable.

Furthermore, several functions cannot be linearized easily, or linearization would introduce large errors. Then, the AADD library can *split* the affine form to two affine forms. As a simple example, take the *abs* function. It is handled as:

$$\mathrm{abs}(x) = \mathrm{abs}(x)|x < 0 \quad \text{or} \quad \mathrm{abs}(x)|x > 0$$

Note that the above notation is like a Shannon Decomposition for a Boolean Function – we will make use of this in AADDs.

Despite splitting, nonlinear functions are often not handled accurately by Affine Arithmetic, whereas the very simple Interval Arithmetic provides often very accurate results for single operations. Therefore, the AADD library uses a combination of both:

- An enclosing hull defined by min and max is computed by both IA and AA, and the result is
  (enclosing hull by AA) intersect (*enclosing hull by IA)* intersect *(image of function)*
- *Open intervals* are represented as an interval and computed only by IA.

1.5 BDD, BDD^A and AADD

Real numbers allow us to model many problems. However, some problems are more suitable to be handled by discrete models, e.g. by Boolean functions. Efficient representation of Boolean functions are, e.g., Binary Decision Diagrams (BDD[2]). We in particular use Reduced and Ordered BDD that are often also abbreviated ROBDD; we stay however with the shorter abbreviation BDD. (RO)BDDs are directed acyclic graph with two leaf nodes *True (or One)* and *False (or Zero)* that model the possible Boolean outcomes of a Boolean function. The internal nodes refer to decision variables. Depending on the (unknown) value of the decision variable, a path through the tree is chosen that ends in True or False, depending on the Boolean function.

In the AADD library, we extend BDDs in two directions and call them BBD^A:

1) Instead of having simple decision variables, we allow the use of predicates ("path conditions") that are represented by comparisons of AADDs.
2) Leaves can take a value Empty, Zero, One of the type XBool that consists of the elements:
   - Empty – Neither True nor False; e.g., if the path conditions lead to a contradiction,
   - Zero – "False" of Boolean,
   - One – "True" of Boolean,
   - All – Both True and False is possible.

In the following, we explain first BDD, then BDD^A and then AADD.

[2] Bryant, R.E. (2018) Binary Decision Diagrams. In: Clarke E., Henzinger T., Veith H., Bloem R. (eds) Handbook of Model Checking. Springer, Cham. https://doi.org/10.1007/978-3-319-10575-8_7.

Boolean decision variables and the ITE function

BDDs can be represented visually by a graph or by repeated calls of the ITE (if-then-else) function. The ITE function takes three parameters of type Boolean. If the first parameter – the decision variable - is True, then the result is the 2nd parameter, else the result is the 3rd parameter. However, keep in mind that if the value of a decision variable is known, no BDD is needed: the result can be evaluated to either TRUE or FALSE.

$$f(a) = \neg a \\ = ITE(a, false, true)$$

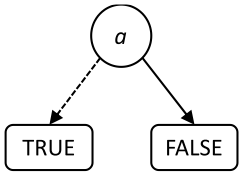


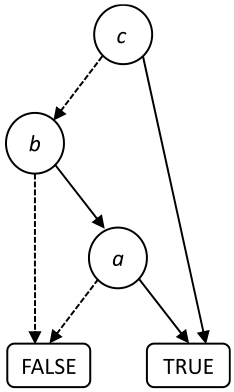
Figure 1-1: BDD of $f(a) = \neg a$ as an ITE function and as a BDD.

Boolean decision variables are just unknown variables. A BDD that models a Boolean function $f(a, b, c) = (a\ and\ b)\ or\ c$ by the ITE function could be written as:

$$f(a, b, c) = ITE(c, True, ITE(b, ITE(a, True, False), False))$$

It would be represented by the following BDD:

$$f(a, b, c) = ab + c$$



AADD provides a very special and basic implementation of Reduced, Ordered BDD: BDD^A
If you need a BDD implementation without interaction to or support for arithmetic operations don't use AADD's BDD, have a look at CUDD and many other "pure discrete" BDD packages.

Conditions: linear constraints as decision variables for BDD^A

Linear constraints are conditions of the type $x > 0$, where x is an affine form. Nonlinear arithmetic expressions are brought to this form as well: every arithmetic expression, in affine arithmetic, results in an affine form, and all kind of comparisons can be brought to the form $x > 0$. (Note, that we do not consider equalities here; equality and Floating-point representations can lead to many issues).

BDD^A

BDD^A are a special kind of BDDs where the decision variables can be linear or linearized constraints, and the leaves can take values from XBool. They are not intended to replace BDDs, but to represent the interaction between predicates that are constraints of non-Boolean values (e.g., x >= 0.0), and the ranges that the non-Boolean values (e.g., x) can take. For example, let there be a Boolean variable c, and let a, b be linear constraints on real-valued functions on $x, y \in \mathbb{R}$; e.g. $a = (h(x, y) > 0)$ and $b = (g(x, y) > 0)$. Then we get for the example above the representation with an ITE function:

$$f(a,b,c) = ITE(c, True, ITE(g(x,y,z), ITE(h(x,y,z), True, False), False))$$

Or, as figure:

$c \in \mathbb{B}; x, y \in \mathbb{R}$

$a = (h(x,y) > 0)$
$b = (g(x,y) > 0)$

$f(x,y,c) = ab + c$

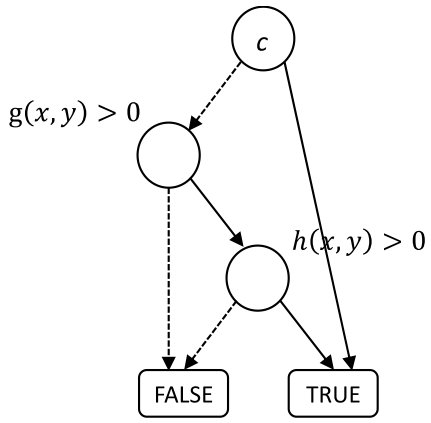


Figure 1-2: BDD^A for the example.

The use of linear constraints as decision variables in BDD^A introduces dependencies between the discrete and the continuous domains: for conventional, discrete BDD, all decision variables are just unknown Booleans. For BDD^A, they can be mixed with linear constraints of type $x > 0$. Such a condition with an affine form $\tilde{x}$ is a linear inequation on the noise symbols as unknowns:

$$x_0 + x_1\varepsilon_1 + \cdots + x_n\varepsilon_n > 0$$

We use the following terms to describe the dependencies between discrete and continuous domains.

- **Condition set** $X_P(v, P)$ of a path P to a leaf v: The set of all conditions on internal nodes on a path from the root of a BDDA to the leaf v, maybe negated, such that v is selected.
- **Path condition** $\chi_P(v, P)$ of a path P to a leaf v: The conjunction of all conditions in $X_P(v, P)$.

Note, that the path condition is not necessarily feasible. As a simple example, consider:

$$f(a) = ITE(a > 2, ITE(a > 1, True, False), True)$$

The condition $a > 2$ is the "highest" node in the BDD^A. The node with the condition $a > 1$ is hence a subtree below the condition $a > 2$. Hence, $a > 1$ will always be True, the leaf with value False will hence never be reached. More general: if there exists an assignment of values to the noise variables such that the path condition can be true, a node is **feasible**. Otherwise, the node is **not feasible** and assigned to the leaf Infeasible that we consider as a "Empty" at leaves. Note, that the above BDD^A for all feasible cases has the result True. It can hence be reduced to a single leaf with the value True.

AADD

AADD are BDD^A where the leaves are affine forms. As an example, let the ITE function be a function that takes AADD for the if/then parameters, and that returns the respective parameter as result. For example, let $\tilde{x} = \varepsilon_1$, then

$$f(\tilde{x}) = ITE(\tilde{x} > 0, \tilde{x} + 100, \tilde{x} - 100)$$

is represented by the following AADD:

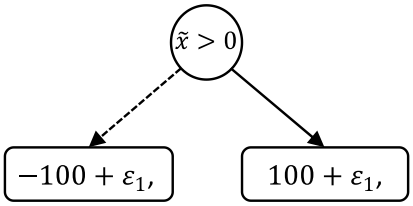


Figure 1-3: AADD for the above expression.

Note that in AADD, each of the conditions in the condition set of a node introduces a linear constraint on the noise variables. Hence, the interval bounds of the affine forms at the leaves are affected: The minimum value of a leaf v with an affine form $\tilde{a} = a_0 + a_1\varepsilon_1 + \cdots + a_n\varepsilon_n$ with path set X_P is now obtained by a linear program:

$$\mathrm{Min}(a_0 + a_1\varepsilon_1 + \cdots + a_n\varepsilon_n)\ s.t.\ X_P\ \wedge\ -1 \leq \varepsilon_i \leq 1 \forall i \in \{1, \ldots, n\}$$

Likewise, for the maximum:

$$\mathrm{Max}(a_0 + a_1\varepsilon_1 + \cdots + a_n\varepsilon_n)\ s.t.\ X_P \wedge\ -1 \leq \varepsilon_i \leq 1 \forall i \in \{1, \ldots, n\}$$

Note, that there exists not necessarily a solution; then, the leaf v is infeasible. Infeasible leaves can be omitted (reduced) as there exists not assignment such that v is reached.

Operations on BDD^A and AADD

Operations on BDD^A resp. AADDs x, y are defined such that for an operation $z = x \odot y$ it holds:

For each leaf of x and y there exists one node in z with:

- $Value(z) = Value(x) \odot Value(y)$
- $X_P(z) = X_P(x) \cup X_P(y)$

Or, short and informally, the result is obtained by computing the operation for each leaf, ensuring that a leaf for each path condition of the operands is in the result.

Relational operations

If we compare an affine form resp. an interval with a real value, we can get the following results:

- True or False, if the value does not lie in the interval. For example, 5 > [1, 2] is for sure *TRUE*, as well as [3,4] > 10 evaluates trivially to *FALSE*; none of these relations needs further variables.
- A new decision variable that is a constraint of kind $\tilde{a} > 0$ if the value lies in the interval. For example, consider the condition [1,3] > 2. Here, the result can be true and false, depending on the noise variables.

In the latter case, the result of a relational operation is a BDD^A that models the two possible Boolean outcomes and its dependency from the comparison.

Likewise, if we compare two AADD, we will get a BDD as for other operations on BDD^A and AADD.

1.6 Modeling dynamic systems with AADD and AADD streams

AADD can be used for various use cases. A particular use case is the symbolic simulation of dynamical systems in different models of computation.

For this use case, we represent signals by bounded sequences of time-tagged values s_t, where the subscript t is from an at least partially ordered set T that models time, and where the values s_t are either real or Boolean values:

$$s_T = \langle s_1, s_2, \ldots \rangle$$

This is a common approach that permits to model systems in different models of computation. If just the (local) order in the sequence is relevant, a system is *untimed*. If T is a discrete ordered set and introduces a global synchronization between two or more signals, the system is a *discrete-time system*. If T is a contiguous subset of the reals, the system is a *continuous-time system*.

To model signals of different kind with AADD, we use *streams* of AADD:

$$\hat{s}_T = \langle \hat{s}_1, \hat{s}_2, \ldots \rangle$$

Now, $\hat{s}_T$ does not represent a single value, but a set of signals (signal-set). The signal-set represents all possible trajectories; by assigning the noise symbols some values, all possible signal trajectories can be obtained (maybe additional by over-approximation). However, $\hat{s}_T$ represents not only the enclosing hull of all reachable signal trajectories.

2 Using AADD from Java or Kotlin

We now explain the use of AADD and BDD to compute arithmetic and Boolean expressions. AADD is provided as a Java archive that can be used from languages that work with the Java Virtual Machine (JVM). These languages include Java, Kotlin, Scala, Groovy and many others.

Furthermore, a shared library version for use with C/C++ or other languages can be generated by the Kotlin Multiplatform.

For using AADD on the JVM platforms, the AADD jar file must be in the classpath of a Java program, and the library and the used classes, methods, fields must be imported.

When using Gradle, use the following dependency (with updated version number >= 0.9):

```
implementation("io.github.tukcps:aadd:$aaddVersion")
```

The following examples use the Java API. For Kotlin, overloaded operators are available that significantly improve readability.

2.1 “Hello world” examples

Hello BDD example in Java and Kotlin

The following example shows how the AADD library can be used in different languages on the JVM platform. We start with Java using the simple example from section 1:

```
import com.github.tukcps.aadd.*;                    // imports DDBuilder


class HelloBDD {
    public void main() {
        DDBuilder builder = new DDBuilder(); // Builder that creates DD
        Boolean c = builder.boolean("c");
        Boolean b = builder.boolean("b");
        Boolean a = builder.boolean("a");
        Boolean f = a.and(b).or(c);          // … compute something
        system.out.println("f(a,b,c) = "+f); // … and print the result
    }
}
```

The resulting output is:

```
f(a,b,c) = ITE(c, true, ITE(b, ITE(a, true, false), false))
```

In the above example, we use the DDBuilder object (a “factory” or “builder” pattern) that creates BDDs. The builder provides methods to create BDD, AADD, and IDD objects.

The DDBuilder object maintains noise symobls and indexes. It is not allowed to apply operations on AADD or BDD from different builders; doing so will throw an exception.

In the example, the method boolean(String: id) of DDBuilder creates an object of the class BDD that represents a Boolean variable with a name given as parameter. As shown above, in Java one must call member functions and, or, etc. of BDD. We can also print a BDD via its member function toString() that is automatically called when used where a String is expected. In the language Kotlin, the same methods can be called in a more readable way by using the overloaded operators and lambdas that implement a DSL (Domain Specific Language):

```
import com.github.tukcps.aadd.*          // import of jAADD

fun main() {
   DDBuilder {                           // DDBuilder is receiver of lambda …
      val c = boolean("c")
      val b = boolean("b")
      val a = boolean("a")
      val f = (a and b) or c             // … and compute something
      println("f(a,b,c) = $f")        // prints result
   }
}
```

The resulting output is for both the Java and the Kotlin code as expected:

Hello AADD example

The use of AADD is like the use of BDD: A builder must be used. In the following we show just the Kotlin example; Java is straightforward using the same functions.

```
import com.github.tukcps.aadd.*

fun main() {
   DDBuilder {
      val x = real(-1.0 .. 1.0, "x")
      val f = ite(x greaterEquals 0.0, x-100.0, x+100.0)
      println(" f = $f")
   }
}
```

In the example,

- The method real(Range, String) creates an AADD object that represents the range using a noise symbol associated with the given string.

- greaterEquals is a comparison operator on AADD whose result is a BDD.

The result is an AADD, as in the example in Section 1:

```
f = ITE(1, [-100,00; -99,00], [99,00; 100,00])
```

The noise terms are not printed by default; instead, the interval computed by the LP solver is printed; the number 1 in the first ITE parameter is the index in which we can access the condition.

In the following, we give examples in Kotlin and leave the translation to Java to the readers.

2.2 DD Builder and AADD operations

In AADD, we consider Intervals and affine forms as a special case of an AADD that is just a single leaf. The class Builder provides methods that allow us to create AADD and BDD.

To get an AADD of kind scalar, that means an affine form

$$\tilde{a} = a_0$$

the method `real(value: Real)` of a DDBuilder instance must be called, e.g.:

```
val scalar = real(a0);    // with a builder as receiver
```

To get an AADD that is represented by an affine form

$$\tilde{a} = a_0 + a_i\varepsilon_i$$

which spans an interval $[a_0 - a_i,\ a_0 + a_i]$ that is dependent on the noise variable with index i, one must use the method real(range: ClosedRange<Real>, id: String) where "id" can be an arbitrary string that uniquely identifies the noise symbol as a human-readable shortcut:

```
val range = real(min .. max, id); // min, max: double, id: String
```

To represent dependency, two AADD must share a noise variable. This can be achieved by giving them the same noise variable's name. AADD that model an interval of kind unbounded in or empty are represented by the following constants:

```
val a = Reals.Empty;    // Empty set; no real number in it.
val b = Reals.All;      // All real numbers, including Infinities, are in it
```

Note, that Empty is a single final instance. All references of it share this single instance that cannot be changed. Cloning Empty will return a reference to the same object again.

Example: *Instantiate AADD of kind scalar with value 1.0 using a noise variable with index 1, the whole set of the Reals, and an empty interval.*

```
import com.github.tukcps.jAADD.*
fun instantiation() {
```

```
    DDBuilder {
        val scalar = real(1.0)
        val range  = real(2.0 .. 3.0, “range”)
        val reals  = Reals.All
        val empty  = Reals.Empty
        println("scalar = " + scalar)
        println("range  = " + range)
        println("reals  = " + reals)
        println("empty  = " + empty)
    }
}
```

The example prints the following output:

```
scalar = 1.0
range  = [2,00; 3,00]
reals  = [-∞; +∞]
empty  = ∅
```

Arithmetic computations with AADD

Objects of the class AADD have methods that permit to do arithmetic operations. For example, to compute the sum of two AADD a, b, i.e. $a = a + b$:

```
a = a + b                          // Java: a.plus(b);
```

Or, for $a = a + b * c/d - e$:

```
a = a + b * c/d-e          // Java: a.plus(b.times(c).div(d)).minus(e);
```

Note that objects from a DDBuilder are by design *immutable*. This means, objects of the class AADD can only be created, but once created, the objects cannot be changed anymore, at least not by arithmetic operations. Each operation on AADD will create as a result a new object of the class AADD, but its parameters are not changed. This allows us to internally work with multiple threads.

However, also note that the variables are *references to objects* that can and will change. For example, if we declare an AADD c, we can assign it an object and later assign c another object. Then, the objects are not changed; they are immutable. But the variable c refers to different objects.

Exercise 1: *Given two intervals $a = [1,2], b = [1,2]$ that are independent which means they use affine forms with different noise symbols. Compute $a - a$ and $a - b$ and print and compare the results.*

```
val a = real(1.0 .. 2.0, "a")
val b = real(1.0 .. 2.0, "b")
```

```
println("    a-a = " + (a - a))
println("but a-b = " + (a - b))
```

The exercise prints the following output:

```
    a-a = [-0,00; 0,00]
but a-b = [-1,00; 1,00]
```

Exercise 2: *What is the max and min volume of an ellipsoid with width/height/depth independently from the range [1, 10]?*

Solution:

$$vol = \frac{4}{3}\,\pi\, a\, b\, c$$

```
fun ellipsoid_excercise() = DDBuilder {
    // Volume of ellipsoid = 4/3 pi a b c where a, b, c=[1,10]
    val a   = real(1.0 .. 10.0, "a")
    val b   = real(1.0 .. 10.0, "b")
    val c   = real(1.0 .. 10.0, "c")
    val pi  = real(3.141, 3.142, "pi")
    val vol = real(4.0/3.0) * pi * a * b * c
    println("Volume = $vol")
}
```

The result computed by AADD is:

```
Volume = [4,19 .. 4189,33]
```

Operations on unbounded and empty intervals

Operations on empty intervals always return an empty interval.

Operations on unbounded intervals return in many cases an unbounded interval. However, there exist operations that return other kind of AADD, e.g., multiplication with 0 returns an AADD of value 0.

2.3 BDD^A Builder and Boolean expressions

For symbolic computation on Boolean variables, we must use its symbolic representation by BDD^A. BDDA are like (RO)BDD but interact with AADD as we will see in the next section. To use it we import it from the AADD library, and we need to use the builder like for AADD:

BDD Instantiation of BDD^A

The AADD Context provides the following methods to create BDD:

```
Boolean constant(Boolean val); // leaf with true resp. false
Boolean variable(String uid);  // internal node, decision variable uid
```

Furthermore, fab.True and fab.False are objects (constants) in the current context for which only one single instance exists. In fact, BDD^A are a kind of Reduced Ordered BDD that, due to the reduction, have at most one True and one False leaf (and leaves Infeasible, NaB depending on the outcomes of the path condition's LP problem)

To create a decision variable with an internal node, the method fab.variable(String uid) shall be used. It creates a BDD with an unknown decision variable that is True resp. False, depending on the variable named varname.

Furthermore, the method [builder.]constant(bool) can be used to create new objects that model just a single Boolean value with one of the values *true* or *false*; it will be one of the leaves True or False.

Example: *Try instantiation of the constants using the method and the constants described. Print the objects.*

Solution (Java, in Class that inherits DDBuilder):

```
void BDDinstantiation() {          // in scope of a DDBuilder!
   BDD a = constant(true);         // gets BDD w/ Boolean true or false
   BDD x = variable("x");          // Unknown decision variable "x"
   System.out.println("a="+a);
   System.out.println("x="+x);
}
```

Boolean expressions and functions

On BDD the usual Boolean operations are available as class methods. Hence, one can evaluate Boolean expressions. Like for AADD, the operations and, or, xor, not, etc. are defined as methods on BDD. Furthermore, on BDD the ITE function is defined. The ITE function takes two parameters: the first parameter is a decision variable. If it is true, then the 2nd parameter is returned, otherwise the 3rd parameter is returned. On BDD it is implemented as a method. The decision variable is then the BDD object itself, the 1st parameter of the method is the result for the case that the decision variable is true, and the 2nd parameter of the method is the result for the case that the decision variable is true.

To compute the expression on Boolean constants and the Boolean variable x from above: d = false && x || true and e = true && x, in Java:

```
BDD d = Boolean.False.and(x).or(Boolean.True);
BDD e = Boolean.True.and(variable("x"));
System.out.println("d=" + d + "\ne=" + e );
```

The resulting output is:

```
d=true
e=ITE("x", true, false)
```

Exercise: *Explain the results.*

2.4 Example: PI control loop

In this section we give a more comprehensive example.

Exercise: *We can model a simple control loop with doubles as follows:*

```
fun PIcontrolDouble() {
   var setVal   = 0.5
   var isVal    = 1.0
   var piOut    = 0.4
   var inVal: Double
   for(int i = 1; i<50; i++ ) {
      inVal = setVal – inVal
      piOut += inVal * 0.05
      isVal = isVal*0.5 + piOut*0.5
   }
}
```

Re-write the PI controller such that we can see all possible is-values for a range from 0 to 2, and also set the PI controller output to a matching initial state.

```
var setVal = range(0.4, 0.6, "setVal")
var isVal  = range(0.9, 1.0, "isVal")
var piOut  = range(0.5, 0.51, "piOut")
var inVal
for (int i = 1; i<50; i++) {
   inval = setval - isval
   piout += inval * 0.05
   isval = isval.times(scalar(.5)).plus(piout.times(scalar(.5)));
}
```

For saving the signals and its visualization, see Section 4.

3 Conditions, Conditional Statements and Iterations

In Section 2 we have computed arithmetic and Boolean expressions. However, the results had no impact on the control flow. We will show in this section how to compute conditions, how to deal with conditional statements and with iterations (loop).

3.1 From decision variables to conditions

Decision variable with unknown value

In a BDD the decision variables are Boolean variables of unknown value. To create a BDD that is not a leaf, one must use the Builder method variable. It adds a decision variable and returns a BDD with an internal node and the leaves true and false. Note, that if the decision variable is known to be true or false, jAADD will return true or false directly, and not a BDD. An example has already been given in the section on BDD. However, why do we add this superscript "^{A}" to BDD? Let's try two examples.

Decision variable that is a condition

If we compare an affine form resp. an interval with a real value, we can get the following results:

- True or False, if the value does not lie in the interval.
- A new decision variable if the value lies in the interval.

The *comparison* of two AADD returns a BDD whose leaves are results of the respective comparisons of the comparison of the two AADD. For example, consider the following Java code:

```
fun comparison() {
   val a = range(1.0, 3.0, 1)
   val b = range(2.0, 4.0, 2)
   val  c = a greaterThanOrEquals b
   println("c = $c or c = ${c.toIteString()}");
}
```

We compare the two AADD a, b that represent intervals [1,3] resp. [2,4]. We the result of the comparison is unknown; the program prints:

```
c = ITE(2, true, false)
```

The "2" in the ITE function refers to the condition. It is just the index of the comparison "a>b".

Now let's slightly modify this example as follows:

```
fun comparison2() {
   val a = range(1.0, 3.0, "a");
   val b = range(2.0, 4.0, "a");
   val c = a greaterThanOrEquals b);
```

```
    println("c=" + c);
}
```

Now, we get the following result:

```
c = false
```

This is because now a and b are affine forms that share a dependency; there is an overlap of the interval, but both affine forms grow similarly: b will always be $a + 1$, and $a > b$ is hence false.

Exercise: *For a, b as declared above, compute the condition $a * b > a + b$ and print the result.*

```
val a: Real = range(1.0, 3.0, "a")
val b: Real = range(2.0, 4.0, "b")
val c = a * b greaterThanOrEquals a + b
println("c="+c)
```

The result is:

```
c = ITE(2, true, false) // 2 is a predicate saved in a Conditions table.
```

3.2 Conditional statements

Imagine a computation as follows, where the result depends on a conditional statement:

```
val a = [-1.0, 1.0];   // let a be a double from this interval ..
if (a <= 0) a = a+2;
else a = a-2;
```

How can we compute the value of a after the conditional statement? The problem is that the condition (a <= 0) cannot be evaluated to True or False. For this purpose, jAADD provides the functions

- IF,
- END,
- ELSE and
- assignS.

For using AADD, this program then can be written as:

```
fun iteExample1() {
    val a: Real = range(-1.0, 1.0, "a");
```

```
    IF(a.le(scalar(0.0))
        a = a.assign(a + 2.0);
    ELSE()
        a = a.assign(a - 2.0);
    END();
    println("a="+a);
}
```

The result we get is:

```
a=ITE(1, [-2,00; -1,00], [1,00; 2,00])
```

Note, that the number 1 is the index of the 1st condition in the ITE example, the comparison a <= 0.0.

In general, to compute a conditional statement with AADD and BDD^A, apply the following steps:

- Replace the keywords if and else with IF and ELSE; add a terminating END() at the end of the if and else block.
- Replace assignments x = f(x, y, …) with x = x.assignS(f(x, y, …))

3.3 Example: water level monitor

In the following we demonstrate the use of jAADD for (very simple) bounded-time model checking:

- First, do a symbolic simulation
- Second, check assertions on the symbolic representation of all possible signals.

A water level monitor can be modelled in discrete time with jAADD. Assume a water tank that can be either filled with inflow or drained with outflow. An automaton sets the rates after a lower threshold 2.0 has been crossed or drained after an upper threshold 10.0 was crossed.

We assume uncertain parameters and initial state:

- The initial level is in [1, 11],
- The outRate is in [-1, -0.6],
- The inRate is in [0.6, 1], and
- The initial rate (inRate or outrate) is unknown.

We can model this as follows in Kotlin and jAADD; in Java, in particular overloaded operators are not available:

```
var outRate: Real = range(-1.0 .. -.6, "outRate")
var inRate: Real  = range(0.6 .. 1.0, "inRate")
var level: Real   = range(4.0 .. 5.0, "initial level")
var rate: Real    = inRate
```

Now, we can make a simple dynamic model that, in discrete time steps of 1 sec, checks if an upper threshold (10.0) or a lower threshold (2.0) is crossed. And, if the threshold is crossed, assign a new value to the rate:

```
var time = 0.0
for (time in 0 .. 30) {
```

```
  assert(wl in 0.99 .. 11.01)
  IF(level greaterOrEquals scalar(10.0));
     rate = rate.assignS(outrate)
  END()
  IF (level LessThan scalar(2.0));
     rate = rate.assignS(inrate)
  END()
  level += rate
}
```

The resulting level continuously stays in the range [1, 11] which already can be seen after the first step with a negligible runtime. One could already stop here, but above we compute 30 steps to analyse scalability. Note, that this example exhibits exponential runtime as it runs into the path explosion problem. Nevertheless, for 30 steps, runtime should be a few seconds, but towards a minute for 100 steps etc.

4 Input/Output

4.1 Conversion to a string

AADD provide infrastructure for conversion to a string. This is the Java method toString.

For example, to convert an AADD a to a String, just call this method:

```
val a: Real;
val s: String = a.toString();
```

As the method “toString” is an overloaded standard-method of Java for conversion to strings, you can directly print strings as follows:

```
println(“a is: ” + a);
```

4.2 Conversion to JSON

JSON is a format for exchanging objects across platforms, e.g. between a Java Backend and a JavaScript frontend. JSON represents the data in an object as a String.

Conversion to JSON can be done by the method toJson.

```
fun JsonExample() {
    val a: Real = range(1.0, 2.0, “a”);
    val s: String = a.toJson(a);
    println(“s = ” + s);
}
```

The result is:

```
s = {
  "index": 2147483647,
  "value": {
    "x0": 1.5,
    "xi": {
      "4": 0.5
    },
    "r": 0.0,
    "min": 1.0,
    "max": 2.0
  },
```

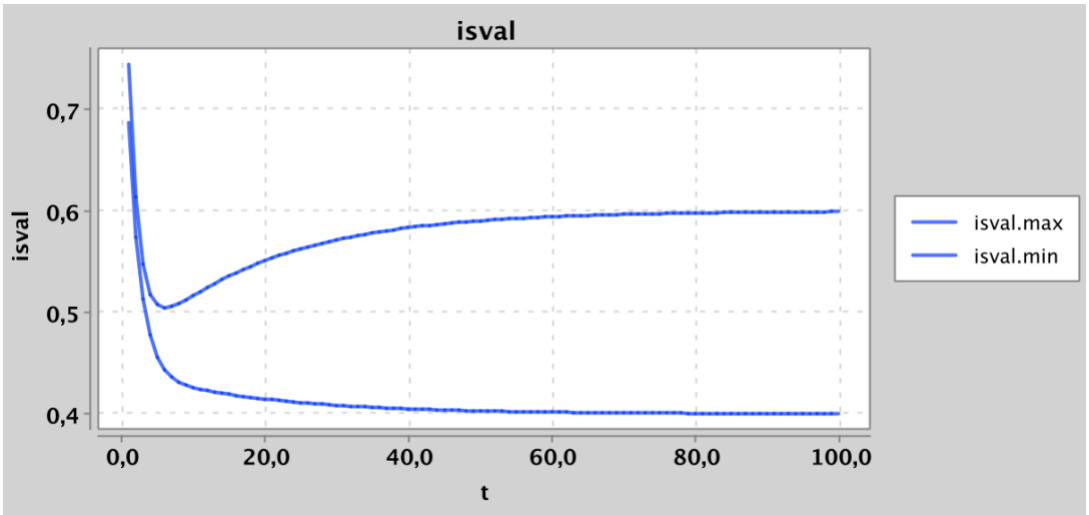
```
  "feasible": true
}
```

4.3 AADD and BDD^A traces

Let's come back to the PI controller example. It would be nice to not only compute the AADDs over time, but to also trace and show the signals over time. In this case, we have t = {1, 2, …, 100}. This can be done via an AADDTrace object.

```
fun piControl() {
  var setVal = range(0.4, 0.6, "setval");
  var isVal  = range(0.9, 1.0, "isval");
  var piOut  = range(0.5, 0.51, "piout");
  var t = new AADDTrace("isval"); // Opens file for tracing
  val inval: Real;
  for (int i = 1; i<50; i++) {
    inval = setval.minus(isval);
    piout = piout.plus(inval.times(scalar(0.05)));
    isval = isval.times(scalar(0.5)).plus(piout.times(scalar(0.5)));
    t.add(isval, i);                    // Adds data to trace
  }
  t.display()
}
```

In a window, the following graph is displayed. It shows the minimum and the maximum values of isval:



The results might be complex, and we might want to save it in a database for later analysis. To write the results in a file, just call this method of AADDstreams:

```
fun toJson(String filename)
```

Annex A: **API of AADD, BDD, Conditions**

A.1 **AADD Library Implementation - Class Diagram**

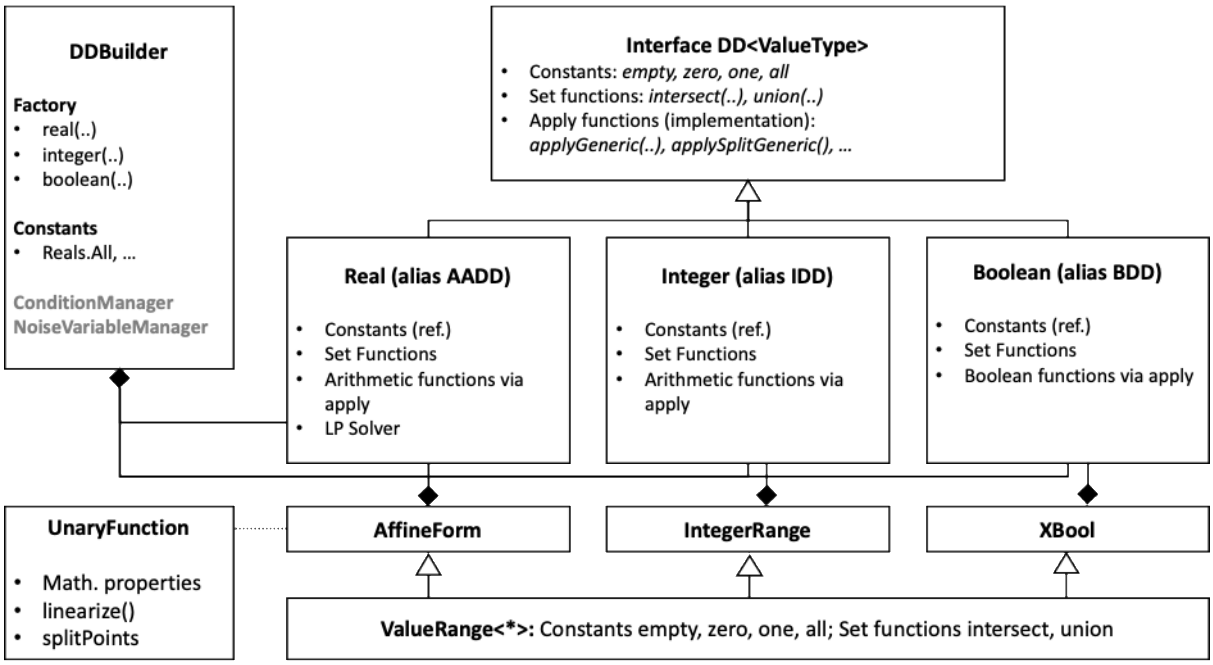


Figure 4-1: Class Diagram

The AADD Library is structured in the following classes.

The main class is the factory:

- DDBuilder, a factory that provides all public methods to create values. Furthermore:
  - Managers for Conditions and Noise symbols.

The following interfaces are implemented by AADD, IDD, BDD:

- ValueRange<*>, the generic interface to model convex sets of values.
  - Declares Empty, Zero, One, All
  - Declares union, intersect
- DD<*>, the generic interface that is the common base class of AADD, IDD, BDD.
  - Has subclasses Internal and Leaf
  - Declares Empty, Zero, One, All
  - Declares union, intersect
  - Implements generic apply function (variants for unary functions, with split, etc.)

The main implementation is then compact:

- AADD/Real, an internal implementation of Affine Arithmetic Decision Diagrams.
  - Implements Empty, Zero, One, All
  - Implements union, intersect
- IDD/Integer, an internal implementation of Integer Decision Diagrams.
  - Implements Empty, Zero, One, All

    - Implements union, intersect
- BDD/Boolean, an internal implementation of BDD[A]
    - Implements Empty, Zero, One, All
    - Implements union, intersect

Arithmetic and Boolean functions are implemented as functions (Kotlin allows it ☺) and used to define and overload operators (+, -, *, /). They are supported by:

- UnaryFunction, an interface that is implemented by all functions. It describes mathematical features of the function that are needed by generic implementation of functions to linearize, split, and approximate them:
    - domain, the domain of the function.
    - image, the image of the function.
    - value(value), a function that maps a single value to a single value.
    - range(range), a function that maps an interval to an interval.
    - curvature(range), a function that determines the curvature of the function .
    - derivative(value), the derivative of the function at a value.
    - inverseDerivative(value),
    - secondDerivativeBound(range), needed for Chebychev/MinMax
    - splitPoints(range), needed for splitting.
    - approximationScheme(range), needed for automatic choice of approximation.

  UnaryFunction is used by the function linearize (Taylor, MinMax/Chebychev) that is used by approximate. linearize is independent from concrete math. Functions (only via using an instance of UnaryFunction.

A.2 DDBuilder Methods

The class AADD extends DD provides the following public methods.

To create (immutable!) values of type Real:

- real(value: Double), creates a scalar value of type Real.
- real(value: ClosedRange<Double>), creates a range of type Real from finite Doubles. Only to be used for tests as it cannot handle all FP traps.
- real(value: ClosedRange<DoubleBound>), creates a range of Real, where values of type DoubleBound may be infinite and takes care of FP traps and rounding etc.

To create (immutable!) values of type Integer:

- integer(value: Long), creates a scalar value of type Long.
- integer(value: ClosedRange<Long>), creates a range of Long. Only to be used for tests as Long cannot handle overflow, representation of infinities, and other limitations of Long.
- integer(value: CloseRange<LongBound>), creates a scalar value of Long.

To create (immutable!) values of type Boolean

- boolean(value: Boolean), creates a Boolean constant.

- `boolean(id: String)`, creates a Boolean variable; note, that the variable does not change, it only decides which of its leaves is true.

A.3 Operations and Functions

The following operations on Real, Integer, Boolean are defined:
For Real and Integer:

- Set functions union and intersect.
- Arithmetic functions for *add, subtract, multiply, divide, and overloaded operators plus, minus, times, div;*
- *exp, ln, log, pow, sqr, sqrt:*
- Trigonometric functions *sin, cos, tan, asin, acos, atan.*

The functions are classified according to the values they process:

```
package io.github.tukcps.aadd.values

sealed interface ScalarValue

sealed interface NumericValue : ScalarValue
sealed interface BooleanValue : ScalarValue

interface RealValue : NumericValue
interface IntegerValue : NumericValue
interface XBoolValue: BooleanValue
interface StringValue : ScalarValue
```

A.4 General public methods

```
package io.github.tukcps.aadd

/**
 * ## DDApi
 * Common interface for all domains in the AADD library.
 * It is leaning towards a bounded lattice and provides a basic, shared
 * infrastructure for all domains of DD:
 * Booleans (BDD), Reals (AADD), Integers (IDD), and maybe more.
 */
interface DDApi<DDType> {

    fun DDType.isEmpty(): Boolean
    fun DDType.isZero():  Boolean
    fun DDType.isOne():   Boolean
    fun DDType.isAll():   Boolean

    // ------------ Set operations -----------
    infix fun DDType.join(other: DDType): DDType
    infix fun DDType.intersect(other: DDType): DDType
    infix fun DDType.contains(other: DDType): Boolean

    // ------------ Comparison ---------------
    infix fun DDType.greaterThan(other: DDType): XBool
    infix fun DDType.greaterThanOrEquals(other: DDType): XBool
    infix fun DDType.lessThan(other: DDType): XBool
    infix fun DDType.lessThanOrEquals(other: DDType): XBool
    infix fun DDType.equals(other: Any?): XBool

    // ------------ Min, Max ----------------
    fun min(vararg values: DDType): DDType
    fun max(vararg values: DDType): DDType
}
```

A.5 Numeric public methods

```
package io.github.tukcps.aadd

/**
 * Operators and functions for Numeric types.
 */
interface ArithmeticApi<DDType: DD<ValueType>, ValueType: NumericValue, Number:
Any> { //, Number: Any>: DDApi<DDType> {

    // ------------ Arithmetic operators --------------
    operator fun DDType.plus(other: DDType): DDType = add(this, other)
    operator fun DDType.plus(other: Number): DDType = add(this, other)
    operator fun DDType.minus(other: DDType): DDType = subtract(this, other)
    operator fun DDType.minus(other: Number): DDType = subtract(this, other)
    operator fun DDType.times(other: DDType): DDType = multiply(this, other)
    operator fun DDType.times(other: Number): DDType = multiply(this, other)
    operator fun DDType.div(other: DDType): DDType = divide(this, other)
    operator fun DDType.div(other: Number): DDType = divide(this, other)
    operator fun DDType.unaryMinus(): DDType = negate(this)

    // ------------ Arithmetic functions ---------------
    fun add(a: DDType, b: DDType): DDType
    fun add(a: DDType, b: Number): DDType
    fun subtract(a: DDType, b: DDType): DDType
    fun subtract(a: DDType, b: Number): DDType
    fun multiply(a: DDType, b: DDType): DDType
    fun multiply(a: DDType, b: Number): DDType
    fun divide(a: DDType, b: DDType): DDType
    fun divide(a: DDType, b: Number): DDType
    fun inv(a: DDType): DDType
    fun negate(value: DDType): DDType
    fun abs(value: DDType): DDType

    fun exp(value: DDType): DDType
    fun ln(value: DDType): DDType
    fun log(value: DDType, base: DDType): DDType
    fun log(value: DDType, base: Number): DDType
    fun pow(value: DDType, exponent: DDType): DDType
    fun pow(value: DDType, exponent: Number): DDType
    fun root(value: DDType, degree: DDType): DDType
    fun root(value: DDType, degree: Number): DDType
    fun sqr(value: DDType): DDType
    fun sqrt(value: DDType): DDType

    // -------- Trigonometric functions (fix: nonsense for Int) ------------
    fun sin(value: DDType): DDType
    fun asin(value: DDType): DDType
    fun cos(value: DDType): DDType
    fun acos(value: DDType): DDType
    fun tan(value: DDType): DDType
    fun atan(value: DDType): DDType
}
```

A.6 Boolean public methods

```
package io.github.tukcps.aadd

/**
 * Operators for Boolean Values
 */
interface BoolApi<DDType>: DDApi<DDType> {
    fun not(a: DDType): DDType
    fun and(a: DDType, b: DDType): DDType
    fun nand(a: DDType): DDType
    fun or(a: DDType, b: DDType): DDType
    fun nor(a: DDType): DDType
    fun xor(a: DDType, b: DDType): DDType
    fun nxor(a: DDType, b: DDType): DDType
}
```

Annex B: **Further Literature on AADD**

[1] C. Zivkovic, C. Grimm: Nubolic Simulation of AMS Systems with Data Flow and Discrete Event Models. In: Design, Automation and Test in Europe (DATE). Mar. 2019, pp. 1457–1462. DOI: 10.23919/DATE.2019.8715278.

[2] C. Zivkovic, C. Grimm, M. Olbrich, O. Scharf, E. Barke: Hierarchical Verification of AMS Systems with Affine Arithmetic Decision Diagrams. In: IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 38.10 (Oct. 2019), pp. 1785–1798. DOI: 10.1109/TCAD.2018.2864238.

[3] C. Grimm, M. Rathmair: Dealing with Uncertainties in Analog/Mixed-Signal Systems (Invited). In: Proceedings of the 54th Annual Design Automation Conference, DAC 2017, Austin, TX, USA, June 18-22, 2017. DOI: 10.1145/3061639.3072949.6